.UBAfiuba
FACULTAD DE INGENIERÍA

Predicción de quiebra de empresas mediante técnicas de machine learning

Ing. Gaspar Acevedo Zain

Carrera de Especialización en Inteligencia Artificial

Director: Esp. Ing. Ariadna Garmendia (FIUBA)

Jurados:

Jurado 1 (pertenencia)
Jurado 2 (pertenencia)
Jurado 3 (pertenencia)

Buenos Aires, marzo de 2026

Resumen

En esta memoria se describe el análisis, diseño y puesta en marcha de un sistema que permite detectar si una empresa puede entrar en quiebra. Este sistema será ofrecido como un servicio con el fin de proporcionar información estratégica a potenciales clientes. De esta manera, clientes como analistas de riesgos, ejecutivos de créditos, entre otros, podrán tomar decisiones críticas en sus negocios.

La resolución de este trabajo fue posible gracias a la implementación de conocimientos aprendidos durante la especialización, tales como técnicas de análisis de datos, modelos de machine learning, automatización de procesos en entornos de MLOps y puesta en producción de modelos mediante APIs.

Agradecimientos

Esta sección es para agradecimientos personales y es totalmente **OPCIONAL**.

Índice general

Resumen I

1. Introducción general 1
1.1. Contexto del trabajo . . . 1
1.2. Estado del arte . . . 2
1.3. Motivación . . . 3
1.4. Objetivos y alcance . . . 3

2. Introducción específica 7
2.1. Técnicas de procesamiento de datos . . . 7
2.1.1. Detección y tratamiento de outliers . . . 7
2.1.2. Escalado de variables . . . 8
2.1.3. Balanceo de dataset . . . 9
2.1.4. Selección de features . . . 9
2.2. Modelos de inteligencia artificial . . . 9
2.3. Optimización de hiperparámetros . . . 10
2.4. Herramientas - Tracking server . . . 10
2.5. Herramientas para automatización . . . 10
2.6. Desarrollo de API y herramientas para su consumo . . . 11
2.7. Uso de contenedores . . . 11

3. Diseño e implementación 13
3.1. Flujo de trabajo . . . 13
3.1.1. Etapas del flujo de trabajo . . . 13
3.1.2. Implementación del flujo de trabajo . . . 14
3.2. Arquitectura del sistema . . . 14
3.2.1. Arquitectura del ambiente local . . . 14
3.2.2. Arquitectura del ambiente dockerizado . . . 15
3.3. Preprocesamiento de datos . . . 16
3.3.1. División del dataset . . . 16
3.3.2. Tratamiento de nulos y outliers . . . 17
3.3.3. Escalado de datos . . . 18
3.3.4. Balanceo del dataset . . . 20
3.3.5. Selección de features . . . 20
3.4. Optimización y entrenamiento de modelos . . . 21
3.4.1. Pipelines y modelos . . . 21
3.4.2. Cross validation e implementación durante optimización . . . 22
3.4.3. Modelo Logistic Regression . . . 22
3.4.4. Modelo SVM . . . 23
3.4.5. Modelo XGBoost . . . 24
3.4.6. Selección del mejor modelo . . . 26
3.5. Tracking server . . . 26

3.6. Automatización mediante Airflow 27
3.6.1. DAG - download and split dataset 27
3.6.2. DAG - model training 28
3.6.3. Configuración mediante YAML 29
3.7. Servicio web 29

Bibliografía **31**

Índice de figuras

3.1. Flujo de trabajo utilizado. 13
3.2. Arquitectura del ambiente local. 15
3.3. Arquitectura del ambiente dockerizado. 16
3.4. Transformación de Yeo-Johnson sobre la columna `Total Asset Turnover`. 18
3.5. Imputación por mediana sobre la columna `Cash Flow Per Share`. 18
3.6. Standard scaler aplicado a `Total expense/assets`. 19
3.7. Standard scaler aplicado a `Retained Earnings to Total Assets`. 19
3.8. Desbalance de la columna Bankrupt? en el dataset de train. 20
3.9. DAG - download and split dataset 27
3.10. DAG - model training 28

Índice de tablas

2.1. Técnicas utilizadas 7

3.1. Técnicas utilizadas 20
3.2. Hiperparámetros - LR 23
3.3. Hiperparámetros - SVM 24
3.4. Hiperparámetros - XGBoost 25

Dedicado a... [OPCIONAL]

Capítulo 1

Introducción general

En este capítulo se introduce el concepto de quiebra empresarial, su distinción con la insolvencia, sus principales causas y repercusiones, y la relevancia de anticiparla mediante modelos predictivos. A continuación, se presenta el estado del arte en predicción de quiebra, junto con las ventajas y limitaciones de las soluciones existentes. Por último, se expone las motivación, el objetivo y el alcance del presente trabajo.

1.1. Contexto del trabajo

La quiebra es un proceso legal y económico en el que entra una empresa cuando es incapaz de cumplir con las obligaciones financieras hacia sus acreedores a medida que vencen, por lo que busca protección o resolución bajo procedimientos legales. Este proceso puede resultar en la reorganización, es decir, en la continuación de operaciones bajo supervisión, o en una liquidación, que consiste en la venta los activos de la empresa junto con su posterior cierre.

Es importante marcar la diferencia entre una empresa insolvente y una en quiebra. La insolvencia es una condición necesaria pero no suficiente para que una empresa entre en quiebra. Una empresa insolvente se caracteriza por tener más activos que pasivos, pero es incapaz de pagar sus deudas a medida que vencen. Sin embargo, una potencial liquidación de estos activos permitiría generar ingresos para pagar a los acreedores. Es decir, la insolvencia hace referencia a una condición financiera que pueder ser temporal e incluso reversible. Por su parte, la quiebra consiste en un proceso legal, al que se llega por la persistencia de la insolvencia. Una característica de las empresas en quiebra es que poseen un patrimonio neto negativo, es decir, poseen menos activos que pasivos, por lo que sus acreedores no recibirían el pago de sus deudas, incluso si la empresa fuese liquidada [1].

Generalmente, la quiebra es el resultado de una acumulación de factores como problemas de flujo de caja, exceso de deuda, mala gestión operativa, saturación del mercado, cambios regulatorios o incluso crisis económicas.

Las repercusiones de la quiebra no solo se ven en el ámbito económico, sino que también en los ámbitos social y legal:

- Económicamente, la quiebra de una empresa podría perjudicar a algunos de sus acreedores, ya que no podrían recibir el pago de sus deudas. Esto a su vez podría generar un efecto dominó sobre estos, provocándoles su insolvencia y/o quiebra. Puede también reducir la productividad económica

de una región y el Producto Bruto Internto (PBI), dependiendo del tamaño e influencia de la empresa afectada.

- Socialmente, el desempleo es la consecuencia más directa de una quiebra. Dependiendo de la cantidad de gente afectada, se podría incluso hablar de una recesión a nivel local.
- Legalmente, los procedimientos pueden durar meses o años, y esto consume recursos que se podrían haber utilizado para pagar deudas. También puede presentar ciertos conflictos de interés entre los acreedores y otros interesados, por ejemplo, al definir quién tiene prioridad sobre el cobro de sus obligaciones.

Es por esto que la predicción de quiebra de una empresa resulta fundamental para distintos interesados. Inversionistas y accionistas podrían decidir si comprar o vender acciones ante una posible pérdida de valor; entidades bancarias evaluarían la calidad crediticia de una empresa y tendrían mejor información para determinar si les otorgan un préstamo o no; algunos proveedores podrían ajustar condiciones de crédito y evitar impagos; los gobiernos podrían diseñar políticas de rescate y apoyo; e incluso la propia gerencia de una empresa podría utilizarla como un sistema de alerta temprana, a fin de implementar medidas de reestructuración o de reducción de costos a fin de evitar la quiebra.

1.2. Estado del arte

Existen diversos estudios, técnicas, mecanismos e incluso soluciones de software que permiten predecir la quiebra de empresas.

Hay estudios que buscan encontrar soluciones generales a esta problemática. Por ejemplo, la fórmula matemática *z-score* de Altman [2] utiliza cinco ratios financieros para obtener la probabilidad de que una empresa quiebre en los próximos dos años. Otras soluciones más modernas, como la propuesta en el paper *Bankruptcy Prediction Using Machine Learning and Data Preprocessing Techniques* [3], proponen el uso de modelos de *machine learning* y técnicas de preprocesamiento de datos para solucionar este problema.

Otros estudios se basan en el análisis de un dataset en particular. Por ejemplo, para el dataset de *Taiwanese Bankruptcy Prediction* [4] hay estudios como *Comprehensive Evaluation of Bankruptcy Prediction in Taiwanese Firms Using Multiple Machine Learning Models*, en donde se exploran distintos modelos de *machine learning*; mientras que en otros, como en *Undersampling bankruptcy prediction: Taiwan bankruptcy data* [5], se estudian los resultados de aplicar ciertas técnicas de preprocesamiento sobre el dataset, más precisamente, técnicas de *undersampling*.

En cuanto a soluciones de software, existen herramientas como Gini Machine [6], una plataforma de análisis predictivo basada en aprendizaje automático, y diseñada bajo el enfoque *no-code*.

Si bien el estado del arte para la predicción de quiebra de empresas presenta diferente soluciones, se observan ciertas desventajas en la mayoría de ellas.

Por ejemplo, en el caso de los estudios y papers, estos son útiles porque describen qué métodos, qué técnicas de procesamiento y qué modelos les permitieron

realizar predicciones y obtener buenos resultados. Sin embargo, ninguno de estos trabajos expone sus modelos mediante una herramienta de acceso público. Reproducir los resultados exige no solo dominio técnico en áreas como ciencia de datos o *machine learning*, sino también la recolección y preparación de datos propios. Esto resulta prohibitivo para todos aquellos usuarios no especializados que podrían beneficiarse de estas herramientas.

Por su parte, Gini Machine resuelve algunas de las problemáticas mencionadas, ya que es una solución *no-code*, es decir, es de uso fácil para aquellos usuarios sin conocimientos programación, de estadística y de *machine learning*. Sin embargo, esta herramienta presenta ciertas desventajas, tales como un alto costo de uso, requerir que el usuario entienda qué datos son importantes para armar un buen dataset, e incluso requiere que la cantidad de datos presentados para armar un modelo sean como mínimo 1000 registros [7].

1.3. Motivación

Tal como se expresó en las secciones anteriores, la predicción de quiebra de una empresa representa información de gran valor para distintos actores del mercado financiero. Contar con esta información de manera oportuna les permitirá a bancos, aseguradoras, fondos de inversión y otros actores del sistema financiero tomar decisiones informadas frente a la inminente quiebra de una compañía bajo análisis.

Si bien hoy existen diferentes soluciones a estas problemáticas, que abarcan desde papers hasta soluciones de software, ninguna de estas ofrece una solución de uso fácil para cualquier tipo de usuario, especialmente para aquellos que no poseen conocimientos técnicos en áreas de matemática, estadística, *machine learning* o incluso programación.

Esto constituye la motivicación principal de este trabajo: la creación y la puesta en marcha de un sistema que permita predecir si una empresa entra en quiebra, priorizando que este sea de fácil acceso y uso para usuarios poco experimentados.

1.4. Objetivos y alcance

El objetivo de este trabajo consiste en satisfacer las necesidades de distintos actores del mercado financiero respecto a la predicción de quiebra de una empresa. Para ello, se diseña, se crea y se expone un sistema que permita realizar este tipo de predicciones. Este sistema se publica mediante un servicio web, de modo que sea de fácil acceso para los interesados. Respecto al diseño, el sistema requiere únicamente el ingreso de los datos de una empresa de su interés. De esta forma se eliminan las desventajas presentes en soluciones actuales, tales como el ingreso de gran cantidad de datos para realizar predicciones, como así también tener conocimiento a nivel técnico de los datos presentados.

Para lograr este objetivo, se definió el siguiente alcance:

- Identificar el set de datos para el problema: en este caso, se utiliza el dataset de origen público *Taiwanese Bankruptcy Prediction* [4].

- Conducir un análisis exploratorio de datos: estudiar el dataset en cuestión es de vital importancia para conocer sus características, distribuciones, y potenciales errores a corregir.
- Realizar un preprocesamiento de datos: aplicar distintas técnicas de eliminación de duplicados, corrección de datos faltantes, corrección de *outliers*, balanceo de datos, selección de *features* más importantes, entre otras, que son de suma importancia para mejorar la calidad del dataset.
- Implementar modelos de *machine learning*: es necesario implementar técnicas robustas, tales como SVM [8], logistic regression [9] y XGBoost [10], que permitan realizar tareas de clasificación. De esta manera, se presentan resultados de calidad al usuario.
- Optimizar hiperparámetros de modelos de *machine learning*: es fundamental refinar detalles relacionados a la configuración de los modelos empleados, a fin que la calidad de los resultados sean superiores.
- Obtener métricas y gráficos de cada modelo: estos les otorgan información extra a los usuarios respecto al contexto de sus predicciones.
- Comparar modelos y seleccionar el óptimo: es fundamental que el sistema le permita al usuario final realizar predicciones mediante el mejor modelo posible. Por ello, se comparan los distintos modelos entrenados y optimizados, a fin de encontrar al mejor de ellos.
- Seguimiento de resultados: al entrenar y comparar distintos modelos, es necesario hacer un seguimiento y una persistencia de ellos, como así también de sus métricas y gráficos. Esto facilita la consulta de esta información en distintas etapas, como así también su disposición para las consultas de los usuarios.
- Automatizar etapas: es de suma importancia que las etapas de análisis, optimización, entrenamiento y seguimiento de modelos sean automatizadas. Esto agiliza el ciclo de vida de la aplicación, permitiendo que las mejoras aplicadas al sistema sean publicadas de forma rápida y sencilla para el consumo de los usuarios.
- Crear un servicio web: mediante una API web, un usuario puede realizar predicciones de quiebra al proveer los datos de una empresa en particular. También puede consultar métricas y gráficas de los modelos involucrados, incluyendo al mejor de ellos.

Por otra parte, dentro de las actividades que quedaron excluidas del alcance, se pueden mencionar:

- Despliegue online: el trabajo realizado se expone como un servicio web que se puede ejecutar de manera local. Es decir, no fue desplegado en plataformas de *cloud computing*, tales como Azure [11], AWS [12], Google Cloud [13], entre otras.
- Creación de interfaz web: el trabajo abarca desde la definición de un dataset, hasta la publicación de un servicio web que permite realizar predicciones de quiebra de una empresa. No se desarrolló un sitio web para consumir este servicio. No obstante, el servicio podrá ser utilizado mediante otras herramientas existentes, tales como cURL [14].

- Comercialización y distribución: el trabajo se limita a la creación y despliegue local del sistema descrito, más no a su comercialización y/o distribución.

Capítulo 2

Introducción específica

En este capítulo se comienza detallando las técnicas de procesamiento de datos utilizadas en el trabajo. Luego, se describen los modelos de *machine learning* entrenados. Posteriormente, se describen las distintas herramientas utilizadas para hacer seguimiento de modelos, para automatizar el procesamiento de datos y entrenamiento de modelos, como así también de aquellas empleadas en el desarrollo de APIs y contenedores.

2.1. Técnicas de procesamiento de datos

En este trabajo se emplearon distintas técnicas de procesamiento de datos, que permitieron detectar y tratar *outliers*, escalar variables, balancear el dataset y seleccionar features. En la tabla 2.1 se presentan las técnicas utilizadas.

TABLA 2.1. Técnicas utilizadas en el procesamiento de datos.

Técnica	Utilizada para
IQR [15]	Detección de *outliers*
Imputación por media [16]	Tratamiento de *outliers*
Imputación por mediana [16]	Tratamiento de *outliers*
Imputación por moda [17]	Tratamiento de *outliers*
Imputación por raíz cuadrada [18]	Tratamiento de *outliers*
Imputación de Yeo-Johnson [19]	Tratamiento de *outliers*
Standard scaler [20]	Escalado de variables
SMOTE [21]	Balanceo de dataset
SMOTE-ENN [22]	Balanceo de dataset
ADASYN [23]	Balanceo de dataset
Información mutua [24]	Selección de features

2.1.1. Detección y tratamiento de outliers

En este tipo de trabajo, en el que se entrenan modelos de modelos de *machine learning*, suelen utilizarse datasets con una gran cantidad de variables (columnas) y datos (registros).

Las variables de estos datasets suelen presentar distintos tipos de distribuciones matemáticas, que deben ser analizadas a fin de descubrir sus características. Entre estas se encuentran los valores atípicos u *outliers*, cuya presencia no solo afecta a la propia distribución, sino también al rendimiento de los modelos.

Por este motivo, uno de los procesamientos que se implementan en este trabajo es el de detectar y tratar *outliers*.

Para detectarlos, se utiliza la técnica de IQR o rango intercuartil [15]. Esta mide la dispersión del 50 % central de los datos, es decir, aquellos que se encuentran entre el percentil 25 (cuartil 1 o Q1) y el percentil 75 (cuartil 3 o Q3), y se define mediante la ecuación 2.1.

$$IQR = Q3 - Q1 \tag{2.1}$$

Una vez definido el rango intercuartil, se consideran *outliers* a aquellos valores que se encuentran 1,5 veces el IQR debajo del cuartil 1, o 1,5 veces el IQR sobre el cuartil 3, tal como se define en la ecuación 2.2.

$$outlier(x) = \begin{cases} \text{No es outlier} & Q_1 - 1,5 * IQR \leq x \leq Q_3 + 1,5 * IQR \\ \text{Sí es outlier} & \text{para todo otro caso} \end{cases} \tag{2.2}$$

Una vez detectados los *outliers*, se imputan mediante distintas técnicas, entre las que se encuentran:

- Imputación por media: reemplaza los valores atípicos con el valor medio de la distribución.
- Imputación por mediana: reemplaza los valores atípicos con la mediana de la distribución.
- Imputación por moda: reemplaza los valores atípicos con la moda de la distribución.
- Transformación por raíz cuadrada: aplica la función de raíz cuadrada sobre la columna entera.
- Transformación de Yeo-Johnson [19]: permite normalizar datos, estabilizar la varianza y reducir el sesgo en una distribución. Es una mejora de la transformación Box-Cox [25], ya que funciona con cualquier número real.

2.1.2. Escalado de variables

Además de los *outliers*, las magnitudes de las variables de un dataset pueden también afectar el rendimiento de los modelos. Por ejemplo, si una variable tiene un rango de valores entre 0 y 100 y otra entre 0 y 1000, el modelo puede interpretar que la segunda es más importante, lo cual podría ser erróneo.

Para solucionar estos inconvenientes se utilizan técnicas de escalado de variables.

En particular, se utiliza la técnica de *Standard Scaler* [20], cuya fórmula se observa en la ecuación 2.3. Esta transforma los datos de una variable para que tengan una media (μ) de 0 y una desviación estándar (σ) de 1, es decir, asemeja la distribución a la de una normal.

$$z = \frac{x - \mu}{\sigma} \tag{2.3}$$

2.1.3. Balanceo de dataset

Al trabajar con problemas de clasificación, los datasets implementados tienen una columna *target* u objetivo, la cual describe a una clase a la cual pertenece cada registro. Una de las particularidades que se pueden presentar es la del desbalance: una clase tiene una cantidad mucho mayor de casos que el resto. Estas se conocen como clases dominantes, y su presencia puede afectar el rendimiento de modelos de *machine learning* al hacer que estos ignoren a las clases minoritarias.

Para solucionar este problema, se utilizan técnicas de sobremuestreo u *oversampling*, las cuales agregan registros extras al dataset con el fin de tener un dataset balanceado.

En este trabajo se utilizan las siguientes técnicas:

- SMOTE [21]: permite generar ejemplos sintéticos en función de un caso de la clase minoritaria y sus vecinos más cercanos.
- ADASYN [23]: consiste en una evolución de SMOTE, la cual genera datos sintéticos en áreas donde la densidad de la clase minoritaria es baja y está rodeada de la clase mayoritaria.
- SMOTE-ENN [22]: consiste en una técnicas que comienza aplicando SMOTE, para luego eliminar ejemplos de clases que difieran de sus vecinos mediante ENN (*Edited Nearest Neighbors*).

2.1.4. Selección de features

Otra particularidad de los datasets utilizados en este tipo de trabajo es la cantidad de variables presentes. Estos valores suelen ser muy variados, ya que hay datasets con menos de 10 variables (como el iris dataset [26], que tiene 6 columnas), mientras que otros pueden tener 100 y más columnas.

Para estos últimos casos resulta conveniente aplicar técnicas de selección de características o *features*. Estas nos permiten filtrar algunas columnas con el fin de quedarse con las más importantes. De esta manera, se elimina cierto ruido de datos pocos relevantes, como así también se reduce el sobreajuste (u *overfitting*) al entrenar modelos.

En particular, en este trabajo se hace uso de la técnica de información mutua o *mutual information* [24]. Esta permite permite cuantificar cuánta información se obtiene sobre la variable objetivo al utilizar una característica en particular, es decir, que tan dependientes son entre sí. A mayor valor, mayor es la información que una *feature* nos proporciona sobre la variable objetivo. Por eso, se busca obtener la información mutua de todas las columnas frente al *target*, luego se hace un ranking ordenando de mayor a menor, y finalmente se selecciona un subconjunto de las mejores variables de dicho ranking.

2.2. Modelos de inteligencia artificial

En los trabajos de clasificación mediante datasets, los primeros pasos y tareas suelen dedicarse a la preparación del dataset, utilizando técnicas como las mencionadas anteriormente. Una vez preparado, los datasets son utilizados como entrada

de distintos algoritmos de *machine learning* con el fin de obtener un modelo. Estos modelos posteriormente nos permiten realizar predicciones o clasificaciones a partir de datos particulares de entrada.

En este trabajo, se implementan los siguientes modelos de *machine learning*:

- Regresión logística o *logistic regression* [9]: permite obtener la probabilidad de pertenencia a una de las clases de estudio. A partir de dicha probabilidad se determina a qué clase pertenece una muestra en particular.
- SVM o *support vector machine* [8]: se caracteriza por buscar un hiperplano que separe las clases de estudio con el margen más amplio posible.
- XGBoost [10]: consiste en un modelo de ensambles basado en árboles de decisión. Estos árboles son entrenados de manera secuencial, especializándose en corregir los errores del árbol anterior. Esto permite que sea un modelo robusto al evitar el sobreajuste.

2.3. Optimización de hiperparámetros

Los modelos de *machine learning* se caracterizan por tener hiperparámetros: variables de configuración que se definen antes de comenzar el entrenamiento. Estas se diferencian de un parámetro ya que controlan como se comporta el algoritmo.

A partir de esto, surge la necesidad de optimizar estos hiperparámetros con el fin de encontrar un equilibrio óptimo entre sesgo y varianza, lo cual a su vez permite evitar el sobreajuste (*overfitting*) o subajuste (*underfitting* del modelo.

En este trabajo se utiliza el *framework* optuna [27]: una biblioteca de código abierto basada en python. Esta utiliza una optimización bayesiana para, en lugar de probar valores al azar, aprender de intentos anteriores y enfocarse en las combinaciones de hiperparámetros más prometedoras, encontrando así las más óptimas.

2.4. Herramientas - Tracking server

Al trabajar con distintos modelos de *machine learning* resulta crucial hacer un seguimiento de los mismos, enfocándose en los datos o datasets de entrada utilizados, los hiperparámetros configurados, las métricas y gráficos obtenidas luego del entrenamiento, entre otras cosas.

En este trabajo se utiliza MLFlow [28], una herramienta de código abierto que nos permite hacer este seguimiento y registro de configuraciones, a fin de utilizar tanto estos valores como los modelos en otras etapas del trabajo.

2.5. Herramientas para automatización

Este trabajo consta de distintas etapas, tales como el preprocesamiento de dataset, entrenamiento de modelos, optimización de hiperparámetros, obtención de métricas, entre otras.

El trabajo realizada en ellas se automatiza mediante la herramienta de código abierto Airflow [29], la cual nos permite programar, automatizar y monitorear los flujos de trabajo o *workflows* mencionados.

Estos *workflows* se organizan mediante DAGs [30], o grafo acíclico dirigido por sus siglas en inglés, los cuales siguen un orden lógico (secuencial o en paralelo) sin bucles para lograr un objetivo. De esta forma, se pueden orquestar tareas y escalar procesos de manera automática.

2.6. Desarrollo de API y herramientas para su consumo

En este trabajo se presenta una herramienta web que permite realizar predicciones sobre la quiebra de una empresa. Esta consiste en un API Rest, una interfaz de programación de aplicaciones cuya arquitectura, basada en REST [31], permite la comunicación e intercambio de datos mediante el protocolo HTTP. Esto permite que el servicio web sea escalable, ya que las distintas peticiones de clientes se pueden distribuir entre distintas instancias de un mismo servicio; y simple, ya que tiene métodos HTTP o *endpoints* para operaciones como predicción, recuperación de datos, etc.;

Para desarrollarlo, se usa el framework FastAPI [32], el cual permite programar las API utilizando código python, brindando simplicidades tales como la documentación automática, el soporte de métodos asincrónicos, validación automática de datos de entrada, entre otras.

Para validar el correcto funcionamiento de esta API se utilizan herramientas como curl [14] y postman [33], las cuales permiten realizar peticiones web a los distintos *endpoints* disponibles, tanto de manera manual como automática.

2.7. Uso de contenedores

Las distintas herramientas mencionadas se pueden utilizar de diferentes maneras. Una de estas es mediante el uso de contenedores.

Un contenedor o *container* es una unidad de software que empaqueta a una aplicación y a todas sus dependencias a fin de que se ejecute de forma rápida en cualquier entorno. Es decir, permite que un software sea portable, ya que se puede utiliar tanto de manera local como en la nube; aislado, ya que varias aplicaciones se pueden ejecutar en el mismo entorno sin afectarse entre sí; y fácil de desplegar.

El uso de contenedores en este trabajo se usan contenedores mediante la herramienta Docker [34].

Capítulo 3

Diseño e implementación

En este capítulo se comienza describiendo el flujo de trabajo utilizado, explicando la diferencia entre los ambientes locales y dockerizados. Luego, se describe la arquitectura de cada uno de estos ambientes. Después, se detallan los pasos realizados para preprocesar datos, como así también para entrenar y optimizar los hiperparámetros de los modelos. Posteriormente, se explica cómo se configuran el *tracking server* y Airflow. Finalmente, se describe el diseño del servicio web que permite realizar predicciones.

3.1. Flujo de trabajo

En este trabajo se utilizó un flujo de trabajo que abarca desde la obtención de los datos base hasta la publicación de un servicio web, y puede ser resumido en gran medida por la figura 3.1.

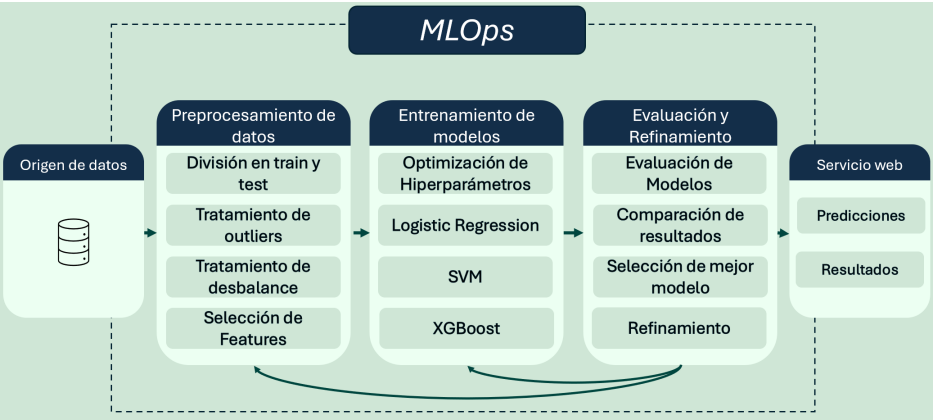

FIGURA 3.1. Flujo de trabajo utilizado.

3.1.1. Etapas del flujo de trabajo

La primera etapa del flujo de trabajo implementado consta de la obtención del origen de datos del sistema: el dataset de *Taiwanese Bankruptcy Prediction* [4]. Este es de origen público y cuenta con más de 6800 registros, que tienen información financiera de empresas del mercado de Taiwán entre 1999 y 2009. La última etapa consiste en la publicación de un servicio web, que permitirá a los distintos usuarios realizar predicciones utilizando un modelo entrenado en este trabajo.

Entre estas dos etapas se encuentran otras que se caracterizan por ejecutarse de manera iterativa. Esto se debe a que se busca mejorar los resultados de las iteraciones anteriores, ya sea modificando algunos de sus parámetros o alterando la cantidad y/o el orden de sub-tareas dentro de cada etapa. Estas son:

- Preprocesamiento de datos: en que, a grandes razgos, se mejora la calidad de dataset. Entre sus sub-tareas se encuentran la división del dataset en train y test, tratamiento de *outliers*, tratamiento de desbalance y selección de *features*.
- Entrenamiento de modelos: en que se crean, entrenan y optimizan modelos de *machine learning*, utilizados para realizar predicciones de quiebra.
- Evaluación y refinamiento de modelos: en que se evalúan los modelos de la etapa anterior y se selecciona al mejor de ellos.

3.1.2. Implementación del flujo de trabajo

Para implementar el flujo de trabajo se optó por definir dos ambientes: un ambiente local y otro dockerizado.

El ambiente local se utiliza principalmente para experimentar distintas técnicas, herramientas e incluso sus posibles combinaciones. Por este motivo, este ambiente está constituído por *notebooks* de python. Una vez determinada cuáles son las mejores técnicas y en qué orden deben ejecutarse, se procede a replicarlas en el ambiente dockerizado.

Por su parte, el ambiente dockerizado está orientado a la automatización del flujo de trabajo. De ahí a que se utilicen distintos contenedores de docker. Estos se crean para las etapas desde la descarga del dataset hasta la exposición de los modelos mediante un servicio web.

3.2. Arquitectura del sistema

En la sección anterior 3.1 se describe el porqué del uso de dos ambientes al implementar el flujo de trabajo. En esta sección se describe la arquitectura de cada uno de estos ambientes. Igualmente, se remarca que en las futuras secciones y capítulos, al hablar de "sistema" se hará referencia al sistema desplegado mediante el ambiente dockerizado.

3.2.1. Arquitectura del ambiente local

El ambiente local se caracteriza por estar compuesto por un total de cinco directorios, cada uno dedicado a explorar distintas técnicas y/o herramientas. Cada directorio tiene una cantidad distintas de notebooks de python, que se ejecutan en el orden indicado en la figura 3.2. Por su parte, las notebooks generan dos datasets, uno para los datos de entrenamiento o *train*, y otro para los datos de pruebas o *test*. Estos datasets son a su vez utilizados como inputs por las notebooks del directorio siguiente en la secuencia de ejecución.

El primer directorio que se tiene en el ambiente local es el de análisis exploratorio. Éste, mediante la única notebook presente, se encarga de descargar el dataset original con el fin de analizarlo. En esta notebook se exploran las *features* del dataset y su columna *target* del dataset, haciendo foco en sus distribuciones, búsqueda de

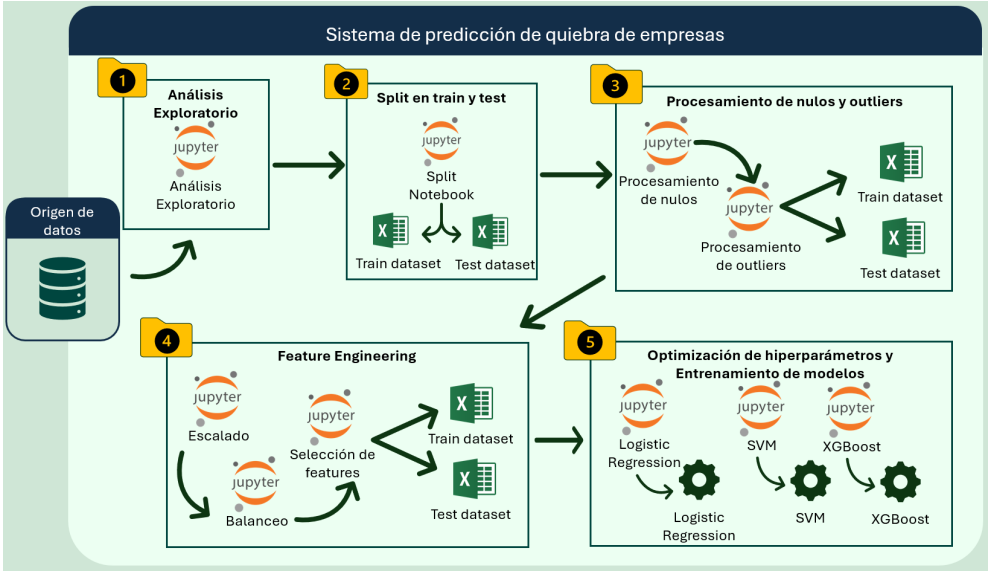

FIGURA 3.2. Arquitectura del ambiente local.

valores nulos y atípicos, como así también en la cantidad de clases presentes en la columna *target*. Como esta es una notebook con fines exploratorios, el dataset de salida es el mismo que el de entrada.

El segundo directorio también consta de una sola notebook. Su objetivo es el de dividir al dataset original en un dataset de *train* y otro de *test*. Estos datasets nuevos constituyen la salida de este directorio, y son utilizados como entrada del siguiente.

El tercer directorio consta de dos notebooks: una para procesar datos nulos y otro para procesar *outliers*. Como los datasets no tienen nulos, los datasets de salida quedan definidos por el notebook de procesamiento de *outliers*.

El cuarto directorio está orientado al *feature engineering*. En este se ejecutan las notebooks de escalado de datos, balanceo de datasets y selección de features de manera secuencial, generando un nuevo dataset de *train* y *test*.

Finalmente, en el quinto directorio se tienen tres notebooks que se ejecutan individualmente. Estas se encargan de optimizar los hiperparámetros y entrenar a los tres modelos de *machine learning* utilizados en este trabajo. Finalmente, cada notebook genera un modelo distinto como salida, que se utilizan localmente para obtener métricas. A partir de estas métricas es que se decide si la ejecución total de notebooks es buena o no, y en caso de serlo, será replicada en el ambiente dockerizado.

3.2.2. Arquitectura del ambiente dockerizado

El ambiente dockerizado consta de cinco módulos principales, entre los que se encuentra el servicio web con el que los usuarios y/o interesados pueden interactuar, ya sea solicitando realizar predicciones o consultando métricas y/o gráficos. En la figura 3.3 se puede esta arquitectura.

El primer módulo en que se realiza la descarga del dataset, la división de *train* y *test*, y el preprocesamiento. Es decir, este módulo abarca las tareas realizadas en los primeros tres directorios del ambiente local.

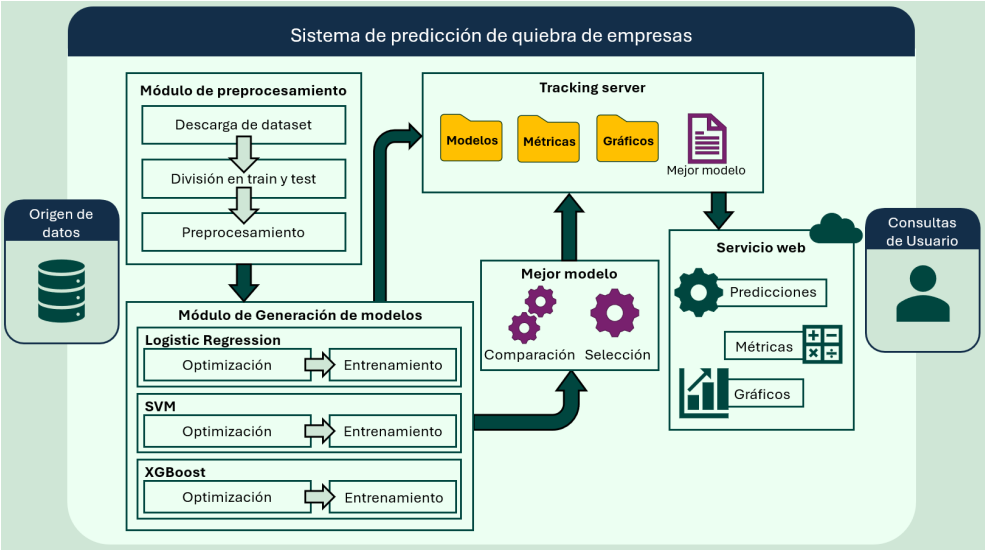

FIGURA 3.3. Arquitectura del ambiente dockerizado.

En el segundo módulo es en el que se realiza la optimización de hiperparámetros y el entrenamiento de los modelos de *machine learning*. Si bien estas dos tareas son secuenciales, las ejecuciones correspondientes a cada modelo se realizan en paralelo. Como salida se obtienen los tres modelos, que son utilizados por los módulos de *tracking server* y "mejor modelo".

El módulo de "mejor modelo", como su nombre lo indica, se encarga de comparar los modelos de *logistic regression*, SVM y XGBoost en función de sus métricas. A partir de esto es que se decide cuál de ellos es el mejor. Este modelo será guardado en el *tracking server*, para que posteriormente puede ser utilizado por el servicio web para realizar predicciones.

El *tracking server* consiste en una implementación de MLFlow. Mediante este, se tiene un servidor en donde se almacenan todas las versiones de los modelos entrenados, sus métricas, sus gráficos, como así también todas las versiones del mejor modelo. De esta manera, se puede acceder a esta información en cualquier momento.

Uno de los componentes que accede al *tracking server* es el servicio web. Más precisamente, accede a la última versión del mejor modelo. También accede a las métricas y gráficos de la última versión de cada modelo entrenado. Esto le permite responder a las solicitudes de usuarios, ya sea para realizar inferencias, como también para responder consultas respecto a métricas y/o gráficas.

3.3. Preprocesamiento de datos

El preprocesamiento abarca las tareas relacionadas a la división de dataset, tratamiento de nulos y de *outliers*, escalado de datos, balanceo de dataset y selección de *features*. A continuación se detalla en que consiste cada una de estas tareas.

3.3.1. División del dataset

El dataset original de *Taiwanese Bankruptcy Prediction* [4] cuenta con un total de 6819 registros, entre los que hay un total de 220 casos de empresas que entran en

quiebra. Si bien estos datos son suficientes para entrenar los modelos de *machine learning* deseados, se presentan dos inconvenientes.

El primero es que hay un desbalance del dataset, siendo la clase objetivo mayoritaria la de empresas que no entran en quiebra. El segundo problema consiste en la necesidad de datos para evaluar qué tan bien generalizan los modelos.

Mediante la división del dataset en *train* o entrenamiento y *test* o evaluación se podrá resolver parte de este problema. Esto permitirá tener un set de datos reservado exclusivamente para la evaluación de modelos. Para ello, se utilizará la función `train_test_split` [35] de la biblioteca `scikit-learn` [36]. Esta función además cuenta con el parámetro `stratify`, que permite dividir una clase en particular proporcionalmente entre los datasets resultantes.

Al implementar la función `train_test_split` se opta por generar un dataset de *test* equivalente al 30 % de los datos originales, es decir, 2046 registros de un total de 6819. A su vez, mediante el parámetro `stratify`, se logra una distribución proporcional de registros de empresas en quiebra, quedando 154 para *train* y 66 para *test*.

3.3.2. Tratamiento de nulos y outliers

En este caso comienza buscando la presencia de datos nulos en los datasets de *train* y *test*. Al analizar todas las columnas de estos dataset, no se encontraron nulos, por lo que tampoco se aplican medidas de corrección.

En cuanto a los *outliers* o datos atípicos, se utiliza la técnica rango intercuartil o IQR para detectarlos. Para tratarlos, se implementan las técnicas de imputación por media, imputación por mediana, imputación por moda, transformación por raíz cuadrada y transformación de Yeo-Johnson. Todas estas técnicas son explicadas en las sección 2.1.

Los cálculos del IQR y de las medidas de tendencia central (media, moda y mediana) se hicieron sobre el dataset de *train*, mas no sobre el de *test*. Luego, en *test* se consideran *outliers* a aquellos valores que no se encuentren en el IQR de *text*, según cada columna, y se imputan con los valores obtenidos en *train*.

Esto es para evitar que haya *data leakage*, es decir, que se filtren datos o características del dataset de *test* en el dataset de *train*. Si esto sucediese, los modelos de *machine learning* entrenados con estos dataset con datos filtrados podrían aprender indirectamente algunas de estas características, lo que evitaría que generalicen correctamente. En cuanto a la transformación por raíz cuadrada y transformación de Yeo-Johnson, estas simplemente se aplican sobre todo el rango de valores de los datasets deseados por separado.

Para definir qué técnica se aplica en cada columna, se opta por primero aplicar las cinco técnicas a cada columna. Luego, para cada técnicas, se calculan cuántos *outliers* hay en la columna modificada. Finalmente, se elige a aquella técnica que haya generado la menor cantidad posible de *outliers*.

Si bien este método ayuda a reducir la cantidad de datos atípicos, estos no serán siempre cero. Por ejemplo, la variable `total asset turnover` se trató con la transformación de Yeo-Johnson, la que redujo los *outliers* de 247 a cero, tal como se ve en la figura 3.4.

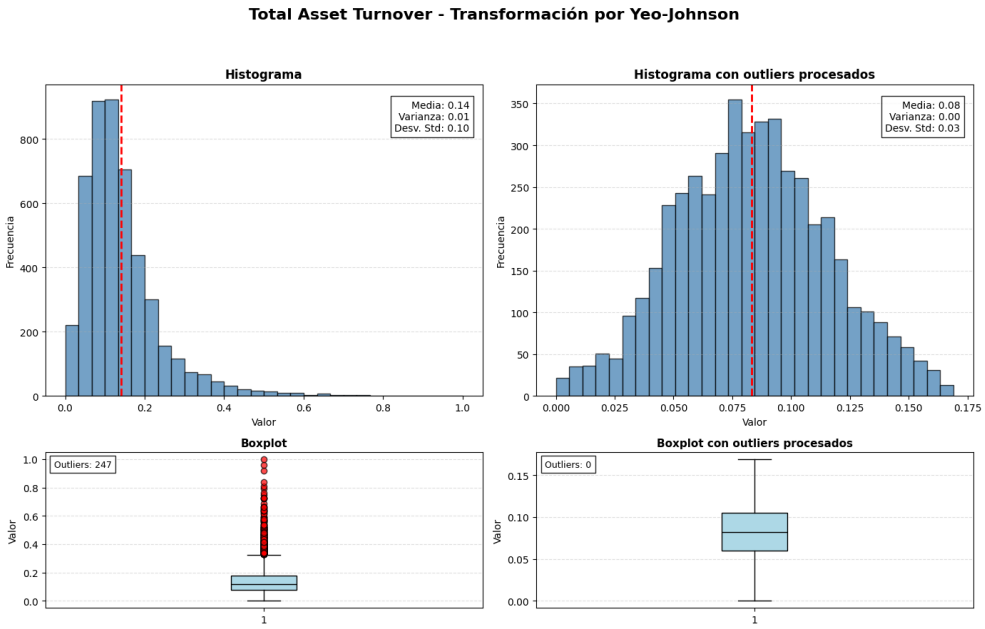

FIGURA 3.4. Transformación de Yeo-Johnson sobre la columna `Total Asset Turnover`.

Pero, para el caso de la variable `cash flow per share`, la técnica que menos *outliers* genera es la de imputación por mediana, tal como se ve en la figura 3.5, la que reduce los *outliers* de 383 a 191.

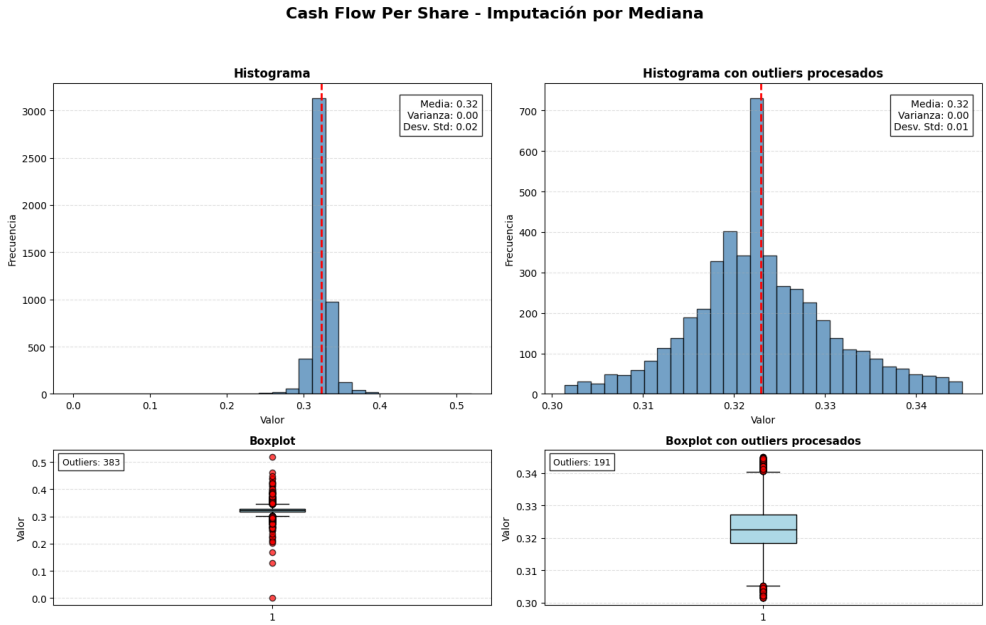

FIGURA 3.5. Imputación por mediana sobre la columna `Cash Flow Per Share`.

3.3.3. Escalado de datos

Las distintas columnas de los datasets presentan diferentes rangos de valores. Esto es un factor que puede perjudicar a los modelos entrenados con dichos datos, ya que pueden priorizar erróneamente una columna frente a otras solamente

por esta diferencia de rangos. Por ello, es fundamental escalar los datos antes de entrenar ciertos modelos de *machine learning*.

Para el escalado de datos se toma una estrategia similar a la del tratamiento de *outliers*: las técnicas de escalado se ajustan en el dataset de *train*, y se aplican en ambos datasets. De esta manera se evita el *data leakage* en esta etapa.

En este caso solo se estudia e implementa la técnica de StandardScaler [20]. El beneficio se encuentra al entrenar modelos de *logistic regression* y SVM, ya que permite mejorar sus reultados. Por su parte, el modelo XGBoost es más robusto a la diferencia de escalas entre columnas. Esto se puede observar en el paper de *The Impact of Feature Scaling In Machine Learning: Effects on Regression and Classification Tasks* [37].

El efecto se puede observar, por ejemplo, en los casos de las columnas `total expense/assets` y `retained earnings to total assets` del dataset de *train*. En la figura 3.6, en el histograma de la izquierda, se observa que la primera de estas columnas tiene un rango relativamente pequeño, que va desde el cero hasta el 0,03, con media de 0,02 y desvío de 0,01.

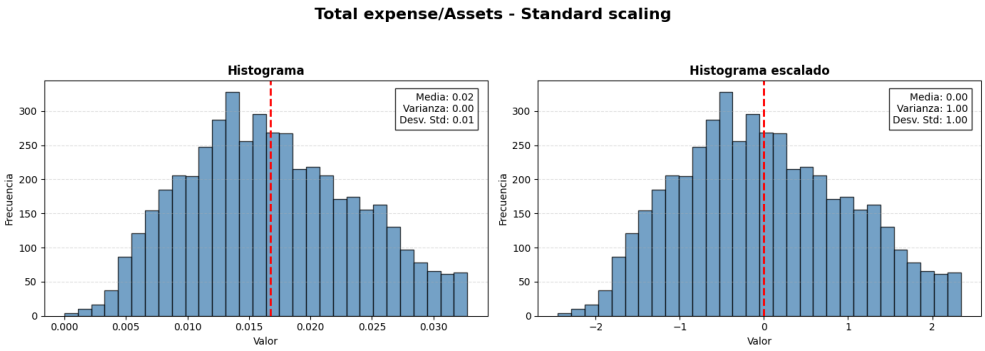

FIGURA 3.6. Standard scaler aplicado a `Total expense/assets`.

Respecto a la columna `retained earnings to total assets`, en el histograma de la izquierda de la figura 3.7, se observa que el rango original es relativamente mayor, ya que va desde el 0,91 hasta el 0,96, con media 0,94 y desvío de 0,01. Al aplicar *standard scaling* a ambas columnas (por separado) se observa que los rangos ahora se encuentran entre -3 y 3 (aproximadamente) para ambos casos, teniendo ambas una media de 0 y un desvío de 1.

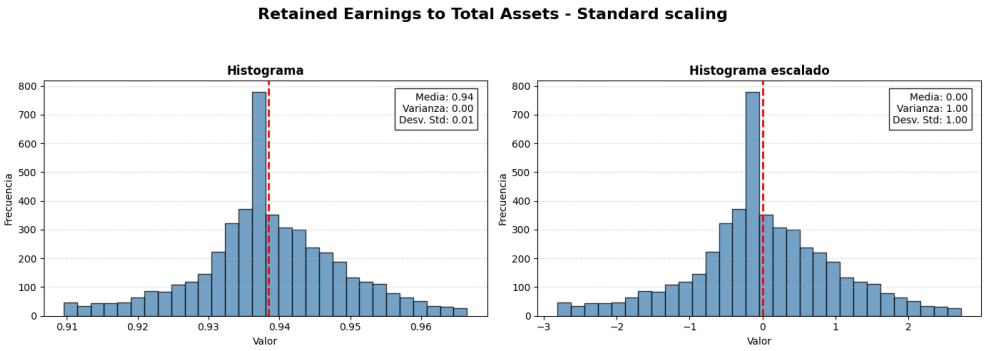

FIGURA 3.7. Standard scaler aplicado a `Retained Earnings to Total Assets`.

3.3.4. Balanceo del dataset

Tal como se menciona en la sección 3.3.1, tanto el dataset original como los dataset resultantes al dividir en *train* y *test* presentan un desbalance en la columna objetivo `Bankrupt`, tal como se observa en la figura 3.8. Este tipo de problemas deben de resolverse antes de entrenar los modelos deseados, ya que si no los modelos podrían generalizar erróneamente nuevos casos.

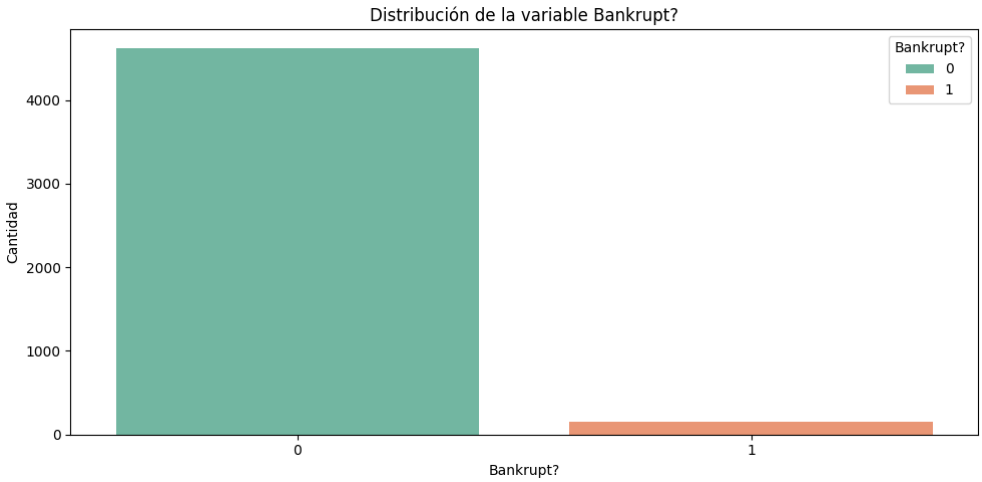

FIGURA 3.8. Desbalance de la columna Bankrupt? en el dataset de train.

Para resolver el desbalance se realizan pruebas con SMOTE [21], SMOTE-ENN [22] y ADASYN [23], técnicas que permiten generar datos sintéticos de distintas maneras, tal como se explica en la sección 2.1.3. Como estos datos son sintéticos, las técnicas se aplican solamente en el dataset de *train*, ya que aplicarlas a *test* podría provocar *data leakage*.

TABLA 3.1. Resultados de las técnicas de balanceo de datos.

Técnica	Registros nuevos	Clases positivas	Clases negativas
ADASYN	9265	4646	4619
SMOTE	9238	4619	4619
SMOTE-ENN	8547	4619	3928

Los resultados de aplicar estas técnicas se observan en la tabla 3.1. En la misma se observa que SMOTE-ENN es la que menos registros generó, como así también la que mayor diferencia entre clases tiene (la clase positiva, o empresas que entran en quiebra, tiene 691 registros más que la otra clase). Igualmente, se opta por aplicar esta técnica en el flujo de trabajo final, ya que es la que mejor resutados arroja.

3.3.5. Selección de features

La última etapa del preprocesamiento consta de la selección de features. Mediante esta, se busca reducir la cantidad de columnas de los datasets, a modo de quedarse con aquellas que mayor información brindan sobre la columna objetivo.

Para este caso, solamente se implementa la técnica de información mutua, explicada en la sección 2.1.4. En las primeras pruebas realizadas mediante esta técnica, se obtuvieron 66 columnas del dataset de *train* que mejor explican a la variable *target*. El resto de las columnas son posteriormente descartados en ambos datasets. Como el número 66 fue elegido al azar (más precisamente, se eligió al azar el 70 % de las columnas más representativas), se opta por configurar este valor representativo de la cantidad de columnas como un hiperparámetro al entrenar los distintos modelos. De esta manera, se obtiene el número que mejor resultados permita obtener en cuanto a predicciones.

Cabe aclarar que otra técnica estudiada es la de ANOVA [38]. Como prerequisito para aplicarla, es necesario que las distribuciones de cada columna sobre las categorías de la columna objetivo tengan distribución normal y sean homocedásticas (es decir, que tengan varianzas iguales). Como estos prerequisitos no se cumplen en el dataset de *train*, se descartó el uso de la técnica de ANOVA para seleccionar features.

3.4. Optimización y entrenamiento de modelos

Una vez preprocesado el dataset original y generados los datasets de *train* y *test*, se comienza con el entrenamiento de modelos. A fin de generar los mejores modelos, se procede a optimizar sus hiperparámetros, como así también se realizan pruebas utilizando *pipelines*.

3.4.1. Pipelines y modelos

En este trabajo se realizaron distintas pruebas con modelos de *machine learning*. En todas estas, el preprocesamiento aplicado era el descrito en la sección 3.3, es decir, una serie de técnicas que, aplicadas sobre el dataset original, generaban un dataset final de *train* y otro de *test*. Luego, estos dataset se utilizan como *input* de los modelos, tanto para entrenarlos como para evaluarlos. Pero ciertos modelos no presentaban buenos resultados. Un ejemplo es XGBoost, que en ciertas pruebas arroja un *recall* de 0,42, un valor muy bajo.

Para mejorar esto, se utiliza la clase `Pipeline` [39] de `Imblearn` [40]. Tal como su nombre lo indica, permite generar un *pipeline* o secuencia de pasos sobre un dataset a fin de ajustarlo, y posteriormente transformar o aplicar los pasos tanto en este como en otros datasets. Las ventajas que poseen estos *pipelines* es que procesan mejor los datos de datasets desbalanceados, manejan mejor el *cross-validation* con *resampling* (técnica que se explica posteriormente) y está específicamente diseñado para usarse al optimizar hiperparámetros.

A nivel implementación, el *pipeline* utilizado consta de los mismos pasos definidos en el preprocesamiento: correción de datos atípicos, escalado, balanceo de datos y selección de features. Se agrega también un paso final correspondiente al entrenamiento del modelo, ya que mejoran los resultados tanto al entrenar como al optimizar hiperparámetros. Luego, mediante el método `fit` junto al dataset de *train*, aplica las transformaciones a los datos y entrena al modelo, mientras que con el método `predict` junto al dataset de *test* evalúa al modelo final.

En cuanto a resultados, la misma secuencia de transformaciones que se utilizaron para el modelo XGBoost y dieron un resultado para *recall* de 0,42, con la implementación de la clase `Pipeline` dieron un *recall* de 0,8. Por este motivo, se opta por utilizar *pipelines* para entrenar y optimizar modelos.

3.4.2. Cross validation e implementación durante optimización

Con el fin de optimizar hiperparámetros se utiliza el *framework* `optuna` [41]. Este se caracteriza por utilizar optimización bayesiana [42], que permite determinar qué combinaciones de parámetros funcionarán mejor basándose en resultados anteriores, lo que permite ahorrar tiempo de cómputo.

Esta herramienta fue combinada con la técnica de *cross validation*, que permite dividir el dataset en k subconjuntos o *folds* iguales, con el fin de realizar k entrenamientos. Más precisamente, durante cada entrenamiento, se utiliza un fold distinto para validar y el resto para entrenar. En particular, se utiliza la función `cross_validate` [43] de `scikit-learn` [36].

Este enfoque asegura que cada configuración de hiperparámetros sea evaluada de manera exhaustiva en diferentes subconjuntos de datos, lo cual a su vez permite prevenir el *overfitting* y mitigar el *data leakage*. También permite obtener una estimación robusta del entrenamiento, ya que se promedian los resultados de los k experimentos.

En este trabajo se opta por utilizar 5 folds, entrenados durante 100 iteraciones de `optuna`. Esto se implementa mediante la clase `StratifiedKFold` [44], que mediante el parámetro `shuffle` permite mezclar los datos para evitar sesgos.

Otra herramienta utilizada es `MedianPruner` [45], que permite realizar una exploración más profunda del espacio de búsqueda en menor tiempo. Esto lo logra al detener automáticamente los ensayos cuyos resultados preliminares, es decir, en durante los primeros folds de la validación cruzada, no superan la mediana de rendimiento de intentos anteriores.

Finalmente, a la hora de optimizar mediante `optuna`, se busca maximizar la métrica de *AUPRC*. Esta métrica es ideal ya que, tal como se menciona en el paper *A closer look at AUROC and AUPRC under class imbalance* [46], es ideal para utilizar en casos donde existe desbalanceo de clases. En particular, esta métrica se obtiene al configurar el parámetro `scoring` de `cross_validate` con el valor `average_precision` [47] (internamente utiliza `average_precision_score` [48]).

3.4.3. Modelo Logistic Regression

El primer modelo que se utiliza en este trabajo es la regresión logística o *Logistic Regression* [9]. Se lo elige principalmente para utilizarlo como modelo *baseline*, es decir, para definir una base de rendimiento antes de utilizar otros modelos más complejos. Igualmente, este modelo presenta ciertos beneficios, como ser computacionalmente económico y rápido para entrenar, o estar diseñado para devolver probabilidades bien calibradas.

Se implementa mediante la clase `LogisticRegression` [49] de `scikit-learn`. En particular, se optimizan los siguientes hiperparámetros:

- `C` o fuerza de regularización: controla el equilibrio entre ajustar bien los datos y evitar el sobreajuste. Mediante `optuna` se explora el rango entre $1e^{-5}$ y $1e^{2}$.
- `Penalty` o penalización: determina que tipo de regularización utilizar. `L1` (Lasso) es excelente para selección de variables, mientras que `L2` (Ridge) es el estándar para estabilidad. Se exploran ambos valores posibles.
- `solver` o algoritmo de optimización: determina el algoritmo a utilizar. Se utiliza `liblinear`.
- `max_iter` o máximo de iteraciones: es el número de pasos que da el algoritmo para converger. Se utiliza el valor 1000.
- `tol` o tolerancia: define el criterio de parada. Si la mejora entre iteraciones es menor a este valor, el modelo deja de entrenar. Ayuda a evitar cómputo innecesario cuando el modelo ya es estable. Se explora el rango entre los valores $1e^{-6}$ y $1e^{-3}$.

Durante la optimización también se busca determinar la cantidad de *features* a elegir para la técnica de información mutua, tal como se expresa en 3.3.5. No solamente se busca cuál es el mejor valor, sino cuál es la mejor manera. Entre estas, se estudian:

- Elección de un porcentaje de *features*: se logra mediante la clase `SelectPercentile` [50]. Para ello se define el parámetro `percentile`, y se explora en el rango 50 y 80, es decir, entre el 50 % y 80 %.
- Elección de una cantidad fija: se logra mediante la clase `SelectKBest` [51]. Se define el parámetro `k_features`, que se explora en el rango 15 y 80.

Los valores encontrados en el proceso de optimización se listan en la tabla 3.2.

TABLA 3.2. Hiperparámetros optimizados para Logistic Regression.

Hiperparámetro	Mejor valor encontrado
`C`	0,0213370341121205
`penalty`	`L1` (Lasso)
`solver`	`liblinear`
`max_iter`	1000
`tol`	0,00013579751380015856
Selección de features	`SelectPercentile`
`percentile`	79

3.4.4. Modelo SVM

El siguiente modelo utilizado es el *Support Vector Machine* o SVM [8]. Este modelo es eficiente en espacios de alta dimensionalidad como el del dataset de estudio, que cuenta con un total de 95 *features*, ya que busca el hiperplano óptimo basándose solo en los puntos más críticos. También se caracteriza por el manejo de relaciones no lineales, lo cual es ideal para este trabajo, ya que la relación entre los ratios financieros y la quiebra no siempre es lineal. Otra ventaja es que es un modelo robusto frente al *overfitting*, ya que maximiza la distancia entre las clases,

lo que tiende a generalizar mejor en datos no vistos. Por último, estudios como *Financial Distress Prediction Using Support Vector Machines and Logistic Regression* [52] remarcan que estos modelos presentan buenos resultados en problemas de predicción/clasifiación de quiebra y/o problemas financieros.

Respecto a su implementación, se utiliza la clase `SVC` [53] de `scikit-learn`, la cual consisten en una implementación de SVM para problemas de clasificación. Esto nos permite explorar los siguientes hiperparámetros:

- `C` o parámetro de regularización: controla el equilibrio entre maximizar el margen del hiperplano y minimizar el error de clasificación en el entrenamiento. Se exploran los valores dentro del rango $1e^{-5}$ y $1e^{2}$.
- `kernel` o función de núcleo: define el tipo de frontera de decisión para transformar los datos. Se exploran los valores `linear`, `poly`, `rbf` y `sigmoid`.
- `gamma` o coeficiente del kernel: define cuánto alcance tiene la influencia de un solo ejemplo de entrenamiento. Solo aplica para kernels `poly`, `rbf` y `sigmoid`. Se estudian los valores dentro del rango $1e^{-4}$ y 1.

De la misma manera que en la sección 3.4.3, se exploran los hiperparámetros para determinar la cantidad óptima de *features* para la selección mediante *mutual information*.

Los mejores hiperparémtros encontrados para SVC se listan en la tabla 3.3.

TABLA 3.3. Hiperparámetros optimizados para SVM.

Hiperparámetro	Mejor valor encontrado
`C`	$0,009090342537573116$
`kernel`	`linear`
`gamma`	$0,0001223840354008705$
Selección de features	`SelectPercentile`
`percentile`	68

3.4.5. Modelo XGBoost

El último modelo estudiado es XGBoost [10]. Este es un algoritmo de *ensemble learning* basado en árboles de decisión que construye modelos de forma secuencial para corregir los errores de los anteriores. Esto hace que, en problemas con datos estructurados (como los de este trabajo), supere consistentemente a otros algoritmos. Otro de los motivos de su elección es que ayuda a prevenir el *overfitting*, ya que incluye penalizaciones L1 (Lasso) y L2 (Ridge) en su función objetivo. Finalmente, este algoritmo presenta el parámetro `scale_pos_weight`, que permite tratar el desbalance al dar más peso a una clase minoritaria, lo cual resulta ideal para el dataset utilizado.

Desde el punto de vista técnico, se utiliza la clase `XGBClassifier` [54] del package `xgboost` [55]. Esta permite utilizar los siguientes hiperparámetros:

- `max_depth` o profundidad máxima: determina qué tan complejo y profundo puede ser cada árbol. Valores altos capturan relaciones más complejas pero aumentan el riesgo de *overfitting*. Se estudia el rango entre 2 y 10.

- `min_child_weight`: define la suma mínima de pesos necesaria en un nodo para crear una nueva partición. Es una de las herramientas más potentes para controlar el sobreajuste: valores altos hacen que el modelo sea más conservador. Se estudian los valores entre 1 y 10.
- `subsample` o submuestreo de filas: fracción de datos de entrenamiento que se muestrea aleatoriamente para construir cada árbol. Ayuda a prevenir el sobreajuste al introducir aleatoriedad. Se estudian los valores entre $0,6$ y 1.
- `colsample_bytree` o submuestreo de columnas: fracción de las 95 variables financieras que se seleccionan para cada árbol. Se estudia el rango entre $0,2$ y 1.
- `learning_rate` o tasa de aprendizaje: controla qué tan rápido aprende el modelo. El rango estudiado es $0,001$ y $0,1$.
- `reg_alpha` o penalización L1: permite regularizar los pesos de las hojas. Se estudia el rango entre 0 y 10.
- `reg_lambda` o penalización L2: permite regularizar los pesos de las hojas. Se estudia el rango entre 0 y 10.
- `n_estimators` o cantidad de estimadores: representa el número total de árboles de decisión que se construirán secuencialmente. El espacio de búsqueda se define entre 100 y 1000.
- `gamma`: especifica la pérdida mínima requerida para realizar una partición adicional en un nodo del árbol. A mayor gamma, más conservador es el algoritmo. Se estudian los valores entre 0 y 5.

De la misma manera que en la sección 3.4.3, se exploran los hiperparámetros para determinar la cantidad óptima de *features* para la selección mediante *mutual information*.

Los mejores hiperparémtros encontrados para XGBoost se listan en la tabla 3.4.

TABLA 3.4. Hiperparámetros optimizados para XGBoost.

Hiperparámetro	**Mejor valor encontrado**
`max_depth`	10
`min_child_weight`	3
`subsample`	$0,6$
`colsample_bytree`	$0,5$
`learning_rate`	$0,1$
`reg_alpha`	$0,1$
`reg_lambda`	5
`n_estimators`	700
`gamma`	$3,9311947393215356$
Selección de features	`SelectPercentile`
`percentile`	75

3.4.6. Selección del mejor modelo

Luego de optimizar los hiperparámetros y entrenar a cada modelo, se obtienen métricas de *recall*, *precision*, *f1-score*, *f2-score*, *AUC-ROC* y *average precision*. Estas se registran en el *tracking server* para poder ser utilizadas en pasos posteriores.

Uno de estos pasos es el de la selección del mejor modelo. El proceso es simple: se ordenan los modelos en función de una métrica configurada, y se selecciona al que presenta mejor valor.

En la configuración actual del trabajo se utiliza la métrica de *average precision*, misma métrica que se usa para optimizar hiperparámetros, la cual pone como mejor modelo a XGBoost.

3.5. Tracking server

En este trabajo se realizaron varias iteraciones sobre sus distintas etapas, principalmente en la de optimización y entrenamiento de modelos (sección 3.4). Como resultado de cada iteración se obtienen: tres modelos nuevos (*Logistic Regression*, SVM y XGBoost), un conjunto de hiperparámetros óptimos para cada modelo, un conjunto de métricas de para cada uno de ellos y, finalmente, un mejor modelo, o modelo ganador/*champion*.

Estos objetos generados son utilizados en etapas posteriores. Por ejemplo, el servicio web utiliza el modelo ganador para realizar predicciones, como así también accede a las métricas de los modelos ante distintas consultas de los usuarios. Por este motivo, resulta fundamental tener un registro de todos los resultados obtenidos.

A partir de esto surge la necesidad de utilizar un *tracking server* que permita almacenar estos recursos, hacer seguimiento de los cambios y comparar los resultados obtenidos.

Por estos motivos se utiliza `MLFlow` [28], una plataforma de código abierto que permite gestionar el ciclo de vida de los modelos entrenados. Esta herramienta cuenta tanto con un *tracking server* [56] como con un *model registry* [57], los cuales pueden ser utilizados mediante un paquete de python. La diferencia entre el *model registry* y el *tracking server* es que este último se enfoca más en la experimentación, mientras que el primero se enfoca más en la gobernanza y producción.

El *tracking server* permite guardar los resultados de cada experimento. Cada experimento se administra con una *run* o ejecución, en donde se guardan los modelos en sí, sus parámetros, sus métricas y algunos gráficos. Esto se logra mediante la clase `mlflow` [58] del paquete python, la cual tiene los siguientes métodos:

- `start_run`: permite comenzar una *run*, como así también grabar cierta información de la misma.
- `log_model`: permite guardar el modelo obtenido en un experimento.
- `log_params`: graba los parámetros utilizados por el modelo en una *run* determinada.
- `log_metrics`: graba las métricas obtenidas por un modelo en una *run* determinada.

- `log_figure`: graba gráficos asociados a un modelo, en una *run* determinada.

Respecto al *model registry*, este permite registrar aquellos modelos que se utilizan en la versión productiva del trabajo, es decor, los mejores modelos o *champion*. Técnicamente, se logra mediante la clase `mlflow.client` [59], con los siguientes métodos:

- `create_registered_model`: crea el registro del modelo.
- `set_registered_model_alias`: permite poner un alias al modelo. En este trabajo el alias para todos los modelos registrados es *champion*.
- `get_registered_model`: permite leer el modelo. Esto resulta ideal para realizar predicciones con el mismo.

3.6. Automatización mediante Airflow

Una vez definidos el preprocesamiento de datos, los modelos a utilizar, sus hiperparámetros y el *tracking server*, se procede a automatizar la interacción entre estos.

Para esto se utiliza `airflow` [29], un paquete de python que actúa como orquestador del flujo de trabajo. Este permite definir DAGs o *directed acyclic graph* [60], un grafo acíclico dirigido donde cada nodo representa las tareas de un determinado flujo, mientras que las aristas marcan el orden de ejecución. En particular, se definieron dos DAGs.

3.6.1. DAG - download and split dataset

El primer DAG definido es `download_and_split_dataset`, el cual se observa en la figura 3.9. Su propósito es el de descargar el dataset *Taiwanese Bankruptcy Prediction* y hacer su división en *train* y *test*.

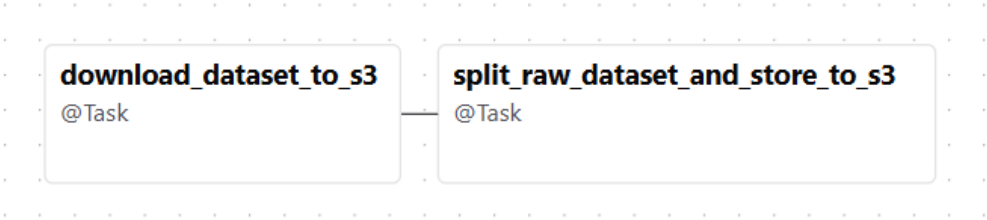

FIGURA 3.9. DAG - download and split dataset

Para esto, se divide el DAG en dos tareas, las cuales se ejecutan secuencialmente:

- `download_and_load_dataset`: descarga el dataset original y lo guarda en una base de datos S3 (el cual está dockerizado).
- `split_raw_dataset`: Toma el dataset original de S3, lo divide en *train* y *test*. Finalmente, guarda estos dos datasets nuevos en S3, para ser utilizados en otros DAGs.

3.6.2. DAG - model training

El segundo DAG es `model_training`, el cual se observa en la figura 3.10. Su propósito es optimizar los hiperparámetros de los modelos, entrenar los modelos y finalmente elegir al mejor de ellos.

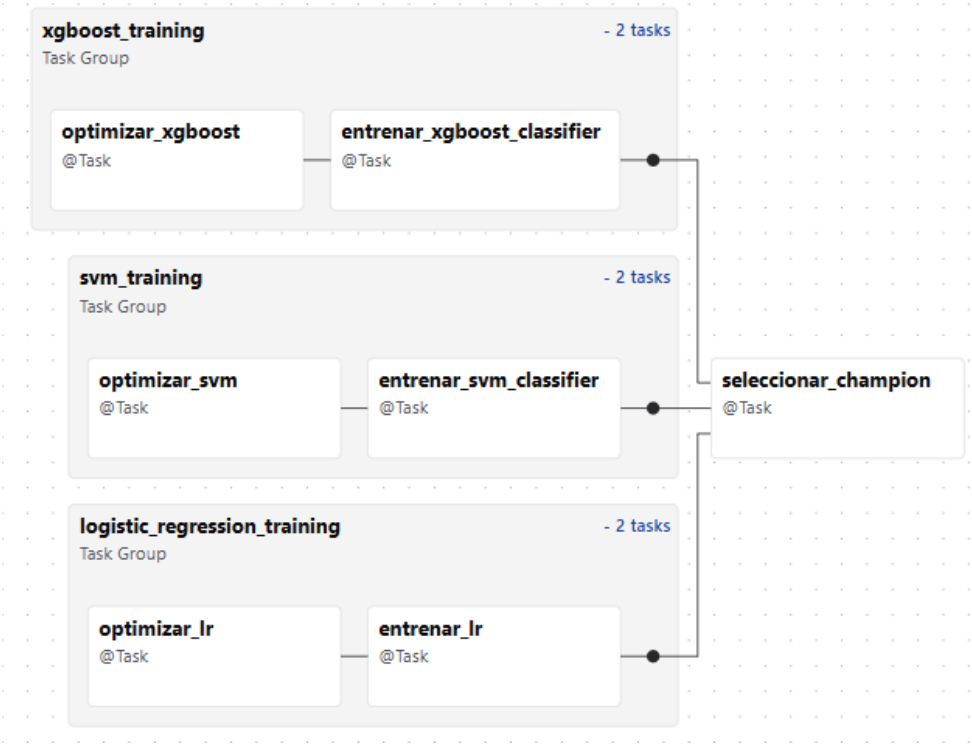

FIGURA 3.10. DAG - model training

Este DAG se caracteriza por ejecutar tres tareas en paralelo, las cuales convergen en una tarea final. Cada una de estas leen los dataset de *train* y *test* originados en el DAG anterior.

Las tareas que se ejecutan en paralelo, que a su vez ejecutan internamente dos tareas secuenciales, son:

- `logistic_regression_training`: se encarga de optimizar los hiperparámetros de *Logistic Regression* (mediante la subtarea `optimizar_lr`) y posteriormente de entrenar dicho modelo (subtarea `entrenar_lr`). Los resultados de cada subtarea se guardan tanto en S3 como en el *tracking server* de `MLFlow`.
- `svm_training`: mediante `optimizar_svm` y `entrenar_svm`, optimiza y entrena un modelo SVM. Sus resultados se guardan tanto en S3 como en el *tracking server* de `MLFlow`.
- `xgboost_training`: de manera similar a las otras tareas de este DAG, tiene una subtarea que optimiza los hiperparámetros (`optimizar_xgboost`) y otra que entrena (`entrenar_xgboost_classifier`) un modelo XGBoost. Los resultados se guardan tanto en S3 como en el *tracking server* de `MLFlow`.

Una vez terminadas, se ejecuta la tarea final `seleccionar_champion`. Esta lee los modelos desde el *tracking server* de `MLFlow`, los compara mediante la métrica

de `average_precision` y al mejor de estos lo guarda en el *model registry* de `MLFlow` como modelo *champion*.

3.6.3. Configuración mediante YAML

Para poder configurar el comportamiento de estos DAGs de `airflow`, se decide utilizar archivos `yaml` [61]. Estos se leen al comienzo de cada tarea del DAG y permiten definir, por ejemplo, qué métrica se utiliza para optimizar hiperparámetros, qué metrica se utiliza para comparar modelos, con qué nombres se guardan los modelos en `mlflow`, entre varios otros detalles.

Uno de las más importantes es la configuración de hiperparámetros de cada modelo. El archivo `yaml` permite pasar valores de pruebas realizadas en otros contextos, por ejemplo, mediante *notebooks* de python. De esta manera, en las subtareas de optimización, se evita ejecutar la optimización en sí y directamente se cargan estos datos. El motivo principal se debe a que, al utilizar `airflow` mediante `docker`, la ejecución de cada DAG demora mucho tiempo. Esto puede deberse a las especificaciones de la máquina con la que se desarrolla este trabajo (Windows 10, 12GB de ram), como así también a cómo `airflow` maneja internamente sus subprocesos.

3.7. Servicio web

El servicio web se implementa mediante el framework FastAPI [32]. El mismo consta de los siguientes endpoints:

- `get_champion_image`: método `get` que permite obtener una imagen asociada al modelo *champion* actual. Las imágenes disponibles corresponden a la matriz de confusión, matriz de confusión normalizada por fila, curva ROC y curva *precision-recall*. Solamente se devuelve de a una imagen por petición.
- `get_champion_metrics`: método `get` que permite obtener las métricas del modelo *champion* actual. Estas métricas incluyen *precision*, *recall*, *f1-score*, *f2-score*, AUC-ROC y *average precision*.
- `get_experiment_image`: método `get` que permite obtener una imagen asociada a cualquiera de las últimas versions de los modelos entrenados, siendo estos *logistic regression*, SVM y XGBoost. Devuelve el mismo tipo de imágenes que el método `get_champion_image`.
- `get_experiment_metrics`: método `get` que permite obtener las métricas de las últimas versiones entrenadas de *logistic regression*, SVM y XGBoost. Son las mismas métricas que devuelve `get_champion_metrics`.
- `predict`: método `post` que permite realizar predicciones a partir de los datos enviados por un usuario.

Internamente, este servicio web utiliza accede al *tracking server* y al *model registry* de `mlflow`. De esta manera, puede utilizar el modelo *champion* para realizar predicciones e incluso para obtener métricas y gráficos. También accede a la última versión entrenada de cada modelo, con el fin de obtener métricas y gráficos. No realiza predicciones con estos.

Respecto al método `predict`, este recibe una petición del usuario con un total de 95 parámetros, correspondientes a cada *feature* del dataset estudiado. Para facilitar la validación de cada una de estas, se utiliza la biblioteca de `pydantic` [62]. Especificamente, se utilizan las clases `BaseModel` [63], que facilita la conversión de una petición de un usuario a una clase propia, y la clase `Field` [64], que permite definir atributos a cada uno de los campos de una clase. Luego, se define una clase propia `CompanyRequest`,que contiene 95 campos que representan las *features* del dataset. Esta clase implementa `BaseModel` para leer fácilmente las peticiones del usuario al método `predict`. Además, mediante `Field`, define los rangos numéricos de cada campo, facilitando sus validaciones.

Bibliografía

[1] Konstantin Danilov. «Corporate Bankruptcy: Assessment, Analysis and Prediction of Financial Distress, Insolvency, and Failure». En: *SSRN Electronic Journal* (sep. de 2014). DOI: 10.2139/ssrn.2467580.

[2] Edward I. Altman. «Financial Ratios, Discriminant Analysis and the Prediction of Corporate Bankruptcy». En: *The Journal of Finance* 23.4 (1968), págs. 589-609. ISSN: 00221082, 15406261. URL: http://www.jstor.org/stable/2978933 (visitado 28-03-2026).

[3] Kamil Samara y Apurva Shinde. «Bankruptcy Prediction Using Machine Learning and Data Preprocessing Techniques». En: *Analytics* 4 (sep. de 2025), pág. 22. DOI: 10.3390/analytics4030022.

[4] Taiwan Economic Journal. *Taiwanese Bankruptcy Prediction*. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C5004D. 2020.

[5] Haoming Wang y Xiangdong Liu. «Undersampling bankruptcy prediction: Taiwan bankruptcy data». En: *PLOS ONE* 16.7 (jul. de 2021), págs. 1-17. DOI: 10.1371/journal.pone.0254030. URL: https://doi.org/10.1371/journal.pone.0254030.

[6] *Gini Machine*. https://ginimachine.com/.

[7] *There is an AI for that - Gini Machine*. https://theresanaiforthat.com/ai/ginimachine/.

[8] Theodoros Evgeniou y Massimiliano Pontil. «Support Vector Machines: Theory and Applications». En: *Support Vector Machines: Theory and Applications*. Vol. 2049. Sep. de 2001, págs. 249-257. ISBN: 978-3-540-42490-1. DOI: 10.1007/3-540-44673-7_12.

[9] Moo K. Chung. *Introduction to logistic regression*. 2020. arXiv: 2008.13567 [stat.ME]. URL: https://arxiv.org/abs/2008.13567.

[10] Tianqi Chen y Carlos Guestrin. «XGBoost: A Scalable Tree Boosting System». En: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '16. ACM, ago. de 2016, 785–794. DOI: 10.1145/2939672.2939785. URL: http://dx.doi.org/10.1145/2939672.2939785.

[11] *Azure*. https://azure.microsoft.com/en-us.

[12] *AWS*. https://aws.amazon.com/.

[13] *Google Cloud*. https://cloud.google.com/.

[14] *cURL*. https://curl.se/.

[15] Vaibhav Tummalapalli. «Outlier Detection and Treatment for Machine Learning Models». En: 2 (nov. de 2025). DOI: 10.5281/zenodo.16500050.

[16] Muhammad M. Seliem. «Handling Outlier Data as Missing Values by Imputation Methods: Application of Machine Learning Algorithms». En: *Turkish Journal of Computer and Mathematics Education (TURCOMAT)* 13 (ene. de 2022), págs. 273-286.

[17] Haneul Lee y Seokheon Yun. «Strategies for Imputing Missing Values and Removing Outliers in the Dataset for Machine Learning-Based Construction Cost Prediction». En: *Buildings* 14.4 (2024). ISSN: 2075-5309. DOI: 10.

3390/buildings14040933. URL: https://www.mdpi.com/2075-5309/14/4/933.

[18] *quare root transformation*. https://medium.com/analysts-corner/square-root-transformation-a-gentle-smoother-for-data-3454233f88a9.

[19] In-Kwon Yeo y Richard A. Johnson. «A new family of power transformations to improve normality or symmetry». En: *Biometrika* 87.4 (dic. de 2000), págs. 954-959. ISSN: 0006-3444. DOI: 10.1093/biomet/87.4.954. eprint: https://academic.oup.com/biomet/article-pdf/87/4/954/633221/870954.pdf. URL: https://doi.org/10.1093/biomet/87.4.954.

[20] *Standard Scaler*. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html.

[21] Nitesh Chawla et al. «SMOTE: Synthetic Minority Over-sampling Technique». En: *Journal of Artificial Intelligence Research* 16 (jun. de 2002), págs. 321-357. DOI: 10.1613/jair.953.

[22] Arnab Mukherjee et al. «SMOTE-ENN resampling technique with Bayesian optimization for multi-class classification of dry bean varieties». En: *Applied Soft Computing* 181 (2025), pág. 113467. ISSN: 1568-4946. DOI: https://doi.org/10.1016/j.asoc.2025.113467. URL: https://www.sciencedirect.com/science/article/pii/S1568494625007781.

[23] Haibo He et al. «ADASYN: Adaptive Synthetic Sampling Approach for Imbalanced Learning». En: *Proceedings of the International Joint Conference on Neural Networks* (jul. de 2008), págs. 1322 -1328. DOI: 10.1109/IJCNN.2008.4633969.

[24] Hanchuan Peng, Fuhui Long y C. Ding. «Feature selection based on mutual information criteria of max-dependency, max-relevance, and min-redundancy». En: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 27.8 (2005), págs. 1226-1238. DOI: 10.1109/TPAMI.2005.159.

[25] G. E. P. Box y D. R. Cox. «An Analysis of Transformations». En: *Journal of the Royal Statistical Society: Series B (Methodological)* 26.2 (1964), págs. 211-243. DOI: https://doi.org/10.1111/j.2517-6161.1964.tb00553.x. eprint: https://rss.onlinelibrary.wiley.com/doi/pdf/10.1111/j.2517-6161.1964.tb00553.x. URL: https://rss.onlinelibrary.wiley.com/doi/abs/10.1111/j.2517-6161.1964.tb00553.x.

[26] *Iris dataset*. https://www.kaggle.com/datasets/uciml/iris.

[27] *Optuna*. https://optuna.org/.

[28] *MLFlow*. https://mlflow.org/.

[29] *Apache Airflow*. https://airflow.apache.org/.

[30] *DAGs*. https://www.astronomer.io/docs/learn/dags.

[31] Carlos Rodriguez et al. «REST APIs: A Large-Scale Analysis of Compliance with Principles and Best Practices». En: jun. de 2016, págs. 21-39. ISBN: 978-3-319-38790-1. DOI: 10.1007/978-3-319-38791-8_2.

[32] *FastAPI*. https://fastapi.tiangolo.com/.

[33] *postman*. https://www.postman.com/.

[34] *Docker*. https://www.docker.com/.

[35] *Función train$_{t}est_{s}plit$*. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html.

[36] *Scikit-learn*. https://scikit-learn.org/stable/index.html.

[37] João Herrera Pinheiro et al. «The Impact of Feature Scaling In Machine Learning: Effects on Regression and Classification Tasks». En: *IEEE Access* PP (ene. de 2025), págs. 1-1. DOI: 10.1109/ACCESS.2025.3635541.

[38] Lars St»hle y Svante Wold. «Analysis of variance (ANOVA)». En: *Chemometrics and Intelligent Laboratory Systems* 6.4 (1989), págs. 259-272. ISSN: 0169-7439. DOI: https://doi.org/10.1016/0169-7439(89)80095-4. URL: https://www.sciencedirect.com/science/article/pii/0169743989800954.

[39] *Imblearn pipelines*. https://imbalanced-learn.org/stable/references/generated/imblearn.pipeline.Pipeline.html.

[40] *Imblearn*. https://imbalanced-learn.org/stable/index.html.

[41] Takuya Akiba et al. *Optuna: A Next-generation Hyperparameter Optimization Framework*. 2019. arXiv: 1907.10902 [cs.LG]. URL: https://arxiv.org/abs/1907.10902.

[42] Gabriele Onorato. *Bayesian Optimization for Hyperparameters Tuning in Neural Networks*. 2024. arXiv: 2410.21886 [cs.LG]. URL: https://arxiv.org/abs/2410.21886.

[43] *cross$_v alidate$*. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.cross_validate.html.

[44] *Stratified K-Fold*. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedKFold.html.

[45] *MedianPruner*. https://optuna.readthedocs.io/en/stable/reference/generated/optuna.pruners.MedianPruner.html.

[46] Matthew B. A. McDermott et al. *A Closer Look at AUROC and AUPRC under Class Imbalance*. 2025. arXiv: 2401.06091 [cs.LG]. URL: https://arxiv.org/abs/2401.06091.

[47] *Scikit-learn - Scoring string names*. https://scikit-learn.org/stable/modules/model_evaluation.html.

[48] *Scikit-learn - average$_p recision_s core$*. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.average_precision_score.html.

[49] *Scikit-learn - LogisticRegression*. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html.

[50] *Scikit-learn - SelectPercentile*. https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.SelectPercentile.html.

[51] *Scikit-learn - SelectKBest*. https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.SelectKBest.html.

[52] Seyyide Doğan, Deniz Koçak y Murat Atan. «Financial Distress Prediction Using Support Vector Machines and Logistic Regression». En: ene. de 2022, págs. 429-452. ISBN: 978-3-030-85253-5. DOI: 10.1007/978-3-030-85254-2_26.

[53] *Scikit-learn - SVC*. https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html.

[54] *XGBoost classifier*. https://xgboost.readthedocs.io/en/stable/python/python_api.html.

[55] *XGBoost package*. https://xgboost.readthedocs.io/en/stable/index.html.

[56] *MLFlow - Tracking server*. https://mlflow.org/docs/latest/self-hosting/architecture/tracking-server/.

[57] *MLFlow - Model registry*. https://mlflow.org/docs/latest/ml/model-registry.

[58] *MLFlow API*. https://mlflow.org/docs/latest/api_reference/python_api/mlflow.html.

[59] *MLFlow client API*. https://mlflow.org/docs/latest/api_reference/python_api/mlflow.client.html.

[60] *Apache Airflow - DAGs*. https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/dags.html.

[61] *YAML*. https://yaml.org/.

[62] *pydantic*. https://pydantic.dev/docs/.
[63] *pydantic - Base model*. https://pydantic.dev/docs/validation/latest/api/pydantic/base_model/.
[64] *pydantic - Field*. https://pydantic.dev/docs/validation/latest/concepts/fields/.